database column type definition.

default: value
: This sets the default value for the column. Since it's performed by the database, you don't see this in a new model object when you initialize it or even when you save it. You have to reload the object from the database to see this value. Note that the default is calculated once, at the point the migration is run, so the following code will set the default column value to the date and time when the migration was run:

```
add_column :orders, :placed_at, :datetime, default: Time.now
```

In addition, decimal columns take the options `:precision` and `:scale`. The `:precision` option specifies the number of significant digits that will be stored, and the `:scale` option determines where the decimal point will be located in these digits (think of the scale as the number of digits after the decimal point). A decimal number with a precision of 5 and a scale of 0 can store numbers from -99,999 to +99,999. A decimal number with a precision of 5 and a scale of 2 can store the range -999.99 to +999.99.

The `:precision` and `:scale` parameters are optional for decimal columns. However, incompatibilities between different databases lead us to strongly recommend that you include the options for each decimal column.

Here are some column definitions using the migration types and options:

```
add_column :orders, :attn, :string, limit: 100
add_column :orders, :order_type,  :integer
add_column :orders, :ship_class, :string, null: false, default: 'priority'
add_column :orders, :amount, :decimal, precision: 8, scale: 2
```

Renaming Columns

When we refactor our code, we often change our variable names to make them more meaningful. Rails migrations allow us to do this to database column names too. For example, a week after we first added it, we might decide that `e_mail` isn't the best name for the new column. We can create a migration to rename it using